IACV Project: Visual reconstruction of played music on a piano keyboard

Veronica Di Gennaro

February 9, 2026

1 Introduction: Problem Formulation and Motivation

The ultimate objective of this work is to identify which keys on a piano keyboard are pressed at any given moment, given a single camera video recording of human hands while playing. The focus on video data is essential, since the proposed solution explicitly relies on temporal analysis of visual changes that cannot be captured from a single still image. The output of the system is a visual augmentation of the original video, where the pressed keys are highlighted while preserving the correspondence between each key and the finger pressing it. This problem is particularly relevant in the context of Music Information Retrieval (MIR), where automatic music transcription aims to convert musical performances into symbolic representations such as MIDI files [1]. While traditional approaches rely mainly on audio signals, they often struggle with polyphonic passages, overlapping notes, or instruments that are not always perfectly tuned. A vision-based approach can complement or, in some cases, replace audio-based methods by observing the physical interaction between the pianist's hands and the instrument. By highlighting pressed keys directly within the video frame, it is possible to maintain information that is often lost in purely audio-based systems. Beyond transcription, this technology has potential applications in the development of automated piano tutoring systems, where visual feedback on incorrect finger placement or timing could assist students during practice sessions. This is particularly relevant for remote learning scenarios, which are increasingly common, especially in underserved areas. From a pure computer vision perspective, where hand detection is a very common and addressed task, piano playing also constitutes a challenging testbed. In piano playing, in fact, the hands can be defined as a "non-collaborative" subject, frequently occluded, and not positioned with the goal of facilitating visual tracking, since a pianist is generally focused entirely on acoustic performance. So finding solutions to this problem can open the door to many other computer vision applications.

2 State of the art

Computer vision research has largely focused on tasks involving hands and fingers including sign language interpretation [9], robotic hand posture learning [3] and other tasks involving gesture recognition [8]. In these contexts, fingers are typically well separated, clearly visible, and intentionally posed to simplify detection reason why classical fingertip tracking approaches based on color segmentation, contour analysis, or decision trees have shown satisfying results. However, the specific context of piano playing introduces significant complexity: pianist fingers are often bent toward the keyboard and frequently touching or overlapping each other. Moreover, fingers may rest on keys without actually pressing them, making the detection of finger-key overlap insufficient to determine which notes are being played. These factors are a clear limit to the effectiveness of the before mentioned conventional hand-tracking solutions, for our specific context. To address these challenges, some researchers in some previous works have proposed multi-camera setups that combine a top-down view with a frontal view to capture the vertical movement of the keys. Other works have alternatively explored the use of depth sensors instead of standard cameras [4], which allow direct measurement of key displacement with a high stability in the detection due to being unaffected by varying skin tones or low contrast between hands and white keys. While these solutions can yield stable results, they require specialized hardware that is not always available, especially for domestic or low-budget use. More recently, Deep Learning techniques, particularly Convolutional Neural Networks (CNNs), have been applied to tasks of hand

pose estimation with high accuracy overcoming the limitations of hand-crafted features. Nevertheless, such solutions often require large, carefully annotated datasets and significant computational resources for training. These requirements make them less suitable for small-scale or exploratory projects such as the present work that will instead focus on a classical computer vision approach that relies only on a single standard camera RGB video and minimal calibration.

3 Proposed Methodology

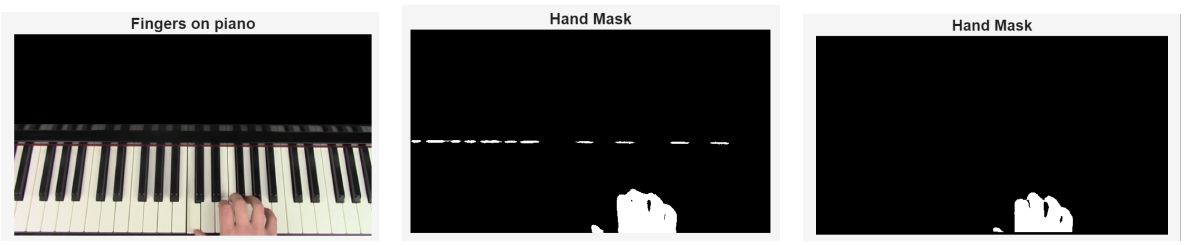
3.1 Solution Approach

The proposed approach is divided into two main components. The first component is purely spatial and aims at identifying candidate keys based on the detected position of the hands and fingers. The second component introduces also a temporal analysis and verifies whether and which of the candidate keys are actually pressed by observing changes in the keyboard over time. The spatial component relies on geometric calibration of the piano keyboard and color-based hand segmentation. By mapping image coordinates to real-world piano coordinates, the keyboard is represented as a virtual set of key regions, enabling reasoning in metric space rather than raw pixel space, which would be less robust. Fingers detected within these regions define a preliminary set of candidate pressed keys. As we anticipated, however, from a top-down view, a pressed key is visually almost indistinguishable from a key touched by a finger but not pressed. To resolve this ambiguity, the temporal component exploits the basic idea that pressing a key introduces structural visual changes, specifically involving the displacement of horizontal boundaries and the deepening of shadows between adjacent keys which are subtle but consistent visual changes that provide a reliable visual cue for confirming which candidate keys are actually pressed.

3.2 Implementation Details

The implementation begins with the mitigation of perspective distortion. Due to the camera viewpoint, that can be located at a slight angle with respect to the keyboard's normal direction, the keyboard may appear as a trapezoidal shape in the image, rather than a rectangle, introducing perspective distortion that complicates direct spatial reasoning. To address this, a Projective Transform is computed by manually selecting the four corners of the keyboard region of interest in the image and mapping them to a known, standard, rectangular representation with real-world dimensions. This transformation allows to convert any image pixel coordinate into a corresponding position on the real-world piano in millimeters, effectively simulating a perfectly top-down perspective view.

The same calibration points are also used to create a binary region-of-interest (ROI) mask that excludes irrelevant image regions such as the surrounding environment or, most importantly, structural elements of the piano, such as the typical felt strip included in many professional piano keyboards to prevent the keys from making a sound when returning to their resting position. This mask reduces false detections and noise especially during hand/skin segmentation as it can be visualized in Figure 1.



(a) Single frame extracted from video taken from [5]. Notice the red felt strip places at the upper end of the keys.

(b) Noisy hand segmentation mask due to the red felt strip being involved in the skin pixels detection process (false positive).

(c) Clean hand segmentation mask due to the skin pixel detection being limited to the keyboard region (no false positives).

Figure 1: Comparison of hand segmentation quality with 1c and without 1b binary region-of-interest (ROI) mask to exclude structural elements of the piano in the segmentation.

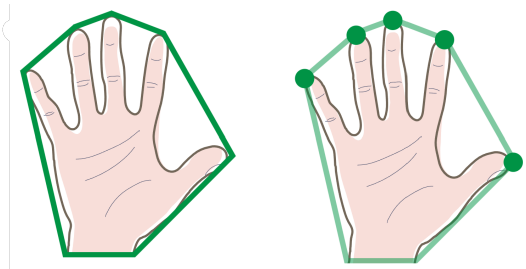
Following calibration, the system pre-calculates a "Virtual Keyboard" using a Cell Array in MATLAB. Each index in the array stores the real-world coordinates of a specific key such that during frame-by-frame processing, the system can determine whether detected fingers overlap with specific keys using this geometric grid without resorting to pixel-based heuristics.

The frame-by-frame processing loop then follows a two-stage strategy composed of identification of candidate keys (3.2.1) followed by confirmation of pressed keys (3.2.2).

3.2.1 Identification of Candidate Keys

First, a YCbCr color filter is applied to detect skin regions. YCbCr filter was chosen because YCbCr color space was experimentally shown to be efficient for the segmentation and detection of skin color in color images by previous studies [7]. In fact, the separation between luminance and chrominance successfully improves robustness against shadows and brightness variations. In particular skin pixels are identified by isolating Cb (Chroma Blue-difference) and Cr (Chroma Red-difference) assuming that skin pixels typically fall within a Cb range of 100-150 and a Cr range of 150-200 as experimentally shown in [7], even if such thresholds can be slightly adjusted according to different exposure conditions.

The identification of candidate keys is then performed through employing two distinct methods in sequence. The primary method makes use of the Convex Hull algorithm, a geometric technique that treats the detected hand mask as a set of points and calculates the smallest convex set that contains them. A simplified diagram of hand convex hull is shown in Figure 2a. By analyzing the peaks along the Y-coordinate of this hull, it can be seen that they correspond to the fingertips, shown in Figure 2b.



(a) Convex hull for the hand.

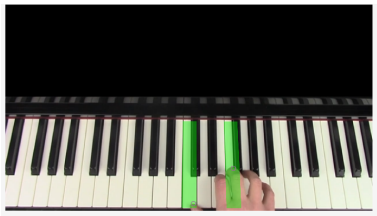
(b) Finger points identified as the extremities of the convex hull.

Figure 2: Both images were taken from [6].

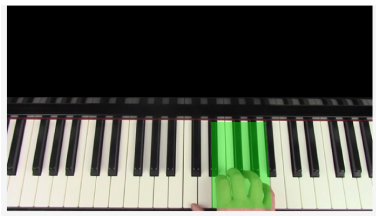
However, this approach encounters some limitations in the context of piano playing, such as overlapping fingers and the fact that typically pianists adopt a curved finger posture. In such cases the hand silhouette becomes more compact making the hull lose the peaks necessary for tip detection. For this reason a secondary, fallback method is integrated. The latter relies, on counting the number of skin pixels within each virtual key region. While this method is more robust during hand overlaps, it cannot function as a standalone solution due to the inherent spatial ambiguity of the human hand on a piano keyboard. For example if the detection threshold for pixel density is set too low, a finger positioned on the edge of one key might erroneously trigger a candidate highlight on its neighbor. On the contrary, if it set too high it fails to detect keys played with only a small portion of the fingertip, as it usually happens when keys are played using the thumb (see Figure 3b or the the pinky finger. In these minimal contact scenarios, the pixel count is then insufficient to cross the fallback threshold, yet the Convex Hull is still capable of identifying the sharp geometric peak of the fingertip, demonstrating the necessity of combining both logics. See Figure 3 for a visual demonstration of this.

3.2.2 Confirmation of Pressed Keys

The final and most crucial stage is confirming which keys are actually pressed, which is achieved by analyzing temporal changes in each candidate key. This analysis is done by generating a temporal difference map by calculating the absolute difference between the current frame and a reference "resting



(a) Only two fingertips (of the thumb and the middle finger) are correctly detected due to the other fingers being curved toward the keys and their fingertips being hidden, causing the convex hull shape getting lost. This results in a incomplete candidate key detection.



(b) The thumb plays the key with a minimal portion of the fingertip and for this reason it is not detected nor classified as a candidate key when only using the pixel count logic.

(c) Combining both convex hull and pixel counting methods, all the keys on top of which there is a finger are correctly classified and highlighted as candidate keys.

Figure 3: Comparison across different methods (3a and 3b) and their combination (3c) for performing candidate key identification for a single frame extracted from video taken from [5]. The keys highlighted in green are those classified by the implemented system as candidate pressed keys.

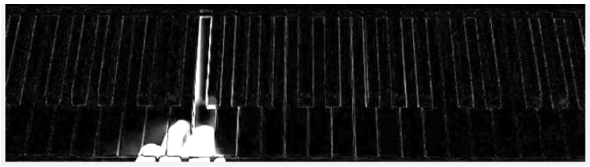


Figure 4: Difference Intensity Map for a pressed key. This demonstrates the intense signal along the horizontal boundary (between black and white keys) and the lateral boundaries of the candidate key. The bright signal intensity at these junctions confirms that mechanical displacement can be effectively captured by the weighted algorithm.

frame" that we select to be the first frame of the input video. While many previous works require a dedicated image of the empty piano, or alternatively having the first frame free of hands, this implementation is designed to work even if the first frame contains hands, provided the keys are at rest.

Ideally, if the intensity change in the candidate key surface exceeds a specific threshold (also this is a modifiable parameter according to lighting and shadow conditions), it indicates that the key has physically moved and cast a new shadow, which confirms an active press as we wished.

To enhance the precision of this stage, the verification logic has been refined from a simple average to a weighted analysis. In fact, instead of looking at the mean intensity change by treating all pixels within the key area equally, a non-homogeneous weigh matrix is generated for each candidate.

This approach allows us to concentrate on specific geometric features of the key surface that are most indicative of a key press (refer to Figure 4):

- **Horizontal Boundary between White and Black keys**: A maximum weight (factor of 15) is assigned to the horizontal boundary between white and black keys (assumed to be located around 90 mm from the top of the white key). This specific "strip" provides the highest signal-to-noise ratio, as the vertical displacement of the key creates a sharp intensity signal.
- **Lateral Boundary between Adjacent Keys**: A secondary weight (factor of 5) is applied to the vertical edges of the key, identified via Canny edge detection. This allows the system to capture intensity shifts relative to adjacent keys.

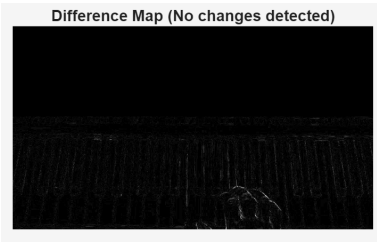
All the remaining visible pixels of the key surface are instead assigned a unitary weight, provided they are not occluded by the pianist's hand (managed via the intersection with the hand mask).

By calculating the weighted mean difference

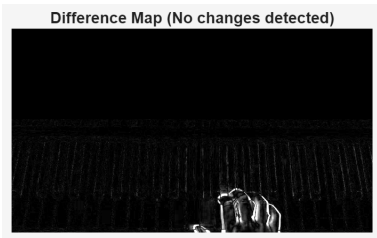
$$\mu_w = \frac{\sum_{i,j}(D_{i,j} \cdot W_{i,j})}{\sum_{i,j} W_{i,j}}$$

where $D_{i,j}$ is the difference map and $W_{i,j}$ is the weight matrix, and comparing it against a tuned threshold, we can distinguish between actual mechanical movement of the key and environmental noise, such as moving shadows or reflections, which would instead be observed if we limited our analysis to simply looking at the mean intensity change across the entire key surface.

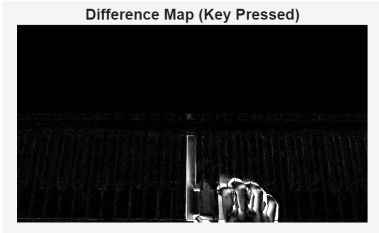
Furthermore, the system implements a Dynamic Fallback Strategy to handle occlusions. In scenarios where the pianist's hand posture obscures the primary horizontal signal, the algorithm automatically shifts the Region of Interest (ROI) toward the proximal portion of the key. In this fallback mode, the weighted average is recalibrated to focus on shadows and intensity variations in this upper part of the white key, ensuring detection stability across diverse playing styles and hand positions.



(a) Difference map in a resting state (or in a state with no changes with respect to the resting state) shows a nearly uniform black field, where the only visible variations consist of minor pixel-level noise caused by subtle light fluctuations and sensor gain, indicating no structural changes to the piano surface.



(b) During hand movement across the keyboard, the difference map highlights the moving hand, but the underlying areas corresponding to the key surfaces, including the upper "trigger zones" remain dark after the skin mask is subtracted, not classifying any key as active press.



(c) The difference map reveals a significant intensity variation within the upper part of the white key signifying that the key has physically moved from its reference position, leading to a successful pressed key detection.

Figure 5: The analysis of the difference map, serves as the primary diagnostic tool for distinguishing between mechanical key displacement and simple hand hovering on top of keys.

4 Experiments and Results

The experimental activity started with the acquisition of the visual data necessary to validate the proposed system: video recordings of piano playing, acquired from a predominantly top-down perspective, as anticipated by the design assumptions of the pipeline.

To assess robustness and generalizability of the approach, different videos with heterogeneous conditions, including variations in lighting, background complexity, keyboard structure, hand morphology, and playing posture, were collected. The experiments involved five different videos, four collected from publicly available online sources and one recorded by me in a domestic environment. The inclusion of a self-recorded video serves to demonstrate the applicability of the method beyond professionally captured videos.

For clarity and conciseness, only three representative videos are discussed in detail in this report, while the remaining experimental results are provided in the uploaded submission folder. For each selected video, a representative frame together with the output of the system is shown in Figure 6. The characteristics of these three chosen videos are described below:

1. The first video, taken from [5], depicts a standard baby grand piano recorded under good lighting conditions, presenting high contrast between keys and minimal shadowing. The camera is not perfectly aligned with the keyboard top-down perspective, introducing a noticeable projective distortion, which makes this example particularly relevant for validating the calibration and homography estimation stages. Additionally, the pianist's hands adopt an evident "claw" posture, resulting in some occluded fingertips.

2. The second video, taken from [2], shows an electronic keyboard being played, rather than an acoustic piano. Moreover it presents a visually complex background with multiple colors and textures. Some shadows are visible on the keyboard surface, especially in the regions between

black keys. The camera placement is close to an ideal top-down configuration, and the hand posture is relatively standard. This scenario is good to show the system's resilience to background and illumination variations.

3. The third video, domestically recorded, represents the most challenging experimental condition. Both hands are present simultaneously on the keyboard, recorded under poor lighting conditions in a dark environment with significant shadowing. The color of the piano structure is similar to the skin tone of the hands, reducing chromatic contrast. For this reason, manual parameter tuning for hand segmentation was needed. Furthermore, the fingers are relatively thin and occasionally overlap.

All videos were selected such that the first frame corresponds to a reference "resting" condition in which no keys are pressed which is exploited by the system for change detection for the identification of key displacements over time.

For the implemented pipeline, we needed to make assumptions about real physical dimension of the piano keyboard. Fortunately, these dimensions are largely standardized across keyboard instruments so it was possible to keep them fixed across all experiments. The keyboard height was assumed to be 150 mm, while the width of a single white key was set to 23.5 mm.

The number of keys visible in each video, instead, varied depending on the captured portion of the keyboard and was treated as a tunable parameter inside the MATLAB implementation code.

Each experimental session began with a manual calibration phase where the four corners of the active keyboard region were manually selected to initialize the projective transformation between image and world coordinates and the virtual key grid.

Following calibration, the system performs candidate key identification, combining the two methods previously described and producing a set of candidate keys that represent the keys potentially being pressed. Subsequently, a verification stage refines this set by analyzing boundary displacement and shadow variation. These two stages are illustrated in Figure 6, where candidate keys are highlighted in green and verified pressed keys are highlighted in red.

Overall, the presented results demonstrate that the proposed solution is able to identify pressed white keys across a wide range of visual conditions, including projective distortion, cluttered backgrounds, low illumination, and multi-hand interaction. While occasional false positives, especially in candidate set, appear in particularly challenging frames, the verification stage significantly reduces their impact. The candidate detection alone, in fact, presents many fluctuations.

5 Conclusions and Open Problems

In this work, aimed at building a computer vision pipeline for the detection of pressed piano keys from single-camera video the focus was completely placed on classical image processing and geometric reasoning techniques, in line with the scope of the course. The experimental results showed that, under a variety of realistic conditions, the system is capable of identifying candidate keys and correctly discriminating between pressed and non-pressed ones.

At the same time, the experiments highlight some limitations of purely algorithmic approaches. In fact, such methods often require manual tuning of parameters and rely on assumptions about lighting conditions, camera placement, and instrument appearance. As a result, generalization to previously unseen environments or significantly different keyboards remains challenging and still unsolved.

Deep learning methods, in particular, could potentially overcome some of the sensitivity to illumination and appearance variations. However, this would require the creation of a suitably large and annotated dataset of piano performance videos.

Another extension of the current pipeline concerns temporal modeling. Currently, the system performs frame-by-frame detection of pressed keys but introducing key tracking over time would allow the reconstruction of complete note sequences, allowing applications such as automatic music transcription.

Moreover there are some challenges that are still unaddressed by this solution. First, only a limited portion of the keyboard is considered everytime, restricting the detectable pitch range. Second, the current implementation is limited to the detection of white keys. In fact, black keys are more difficult to handle due to their different geometry and positioning, and our solution based on boundary and shadow changes would need a lot of modifications. Furthermore, very rapid passages, that are frequent

in piano playing, may introduce a kind of motion blur degrading the performances of our approach involving difference-based cues.

Some extensions are therefore required to strengthen this computer vision pipeline for pressed piano key detection.

References

[1] Mohammad Akbari and Howard Cheng. Real-time piano music transcription based on computer vision. *IEEE Transactions on Multimedia*, 17(12):2113–2121, August 2015.

[2] Bill Hilton. How to play piano for beginners, lesson 1 —— the piano keyboard, December 2016.

[3] I. Infantino, A. Chella, H. Dindo, and I. Macaluso. A cognitive architecture for robotic hand posture learning. *IEEE Transactions on Systems Man and Cybernetics Part C (Applications and Reviews)*, 35(1):42–52, January 2005.

[4] Akiya Oka and Manabu Hashimoto. Marker-less piano fingering recognition using sequential depth images. In *The 19th Korea-Japan Joint Workshop on Frontiers of Computer Vision*, pages 1–4, 2013.

[5] Bitesize Piano. Twinkle twinkle little star - easy piano tutorial, February 2019.

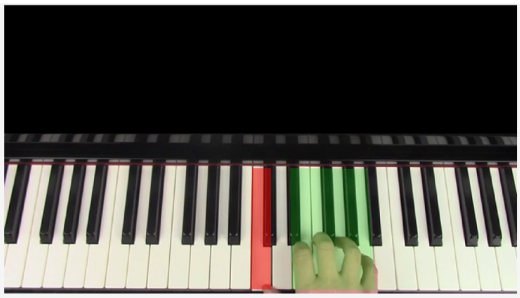
[6] Vangos Pterneas. Finger tracking using kinect v2 – vangos pterneas, January 2016.

[7] Khamar Basha Shaik, P. Ganesan, V. Kalist, B.S. Sathish, and J. Merlin Mary Jenitha. Comparative study of skin color detection and segmentation in hsv and ycbcr color space. *Procedia Computer Science*, 57:41–48, 2015. 3rd International Conference on Recent Trends in Computing 2015 (ICRTC-2015).

[8] Yale Song, David Demirdjian, and Randall Davis. Tracking body and hands for gesture recognition: Natops aircraft handling signals database. In *Proceedings of the 9th IEEE Conference on Automatic Face and Gesture Recognition (FG 2011)*, pages 500–506, Santa Barbara, CA, March 2011.

[9] Jean-Christophe Terrillon, Arnaud Pilpré, Yoshinori Niwa, and Kazuhiko Yamamoto. Robust face detection and japanese sign language hand posture recognition for human-computer interaction in an intelligent room. *Proc. Int'l Conf. Vision Interface*, 01 2002.

(a) Candidate identification on a high-contrast keyboard and a good lightning, no shadows scenario. Fingers are in the "claw position". Candidate keys are highlighted in green.



(b) Verification of actually pressed keys in the same frame as the adjacent figure. Pressed keys are highlighted in red while candidate keys are once again highlighted in green.

(c) Candidate key detection in a complex background scenario. Thanks to the isolation of the candidate regions the system correctly highlights candidate pressed keys.

(d) Pressed key correctly identified.

(e) Candidate key detection in a multi-hand scenario in a dark and shadowy lighting environment, showing the robustness of the pixel-density fallback method in identifying all keys currently under the pianist's hands.

(f) Pressed key correctly identified.

Figure 6: To validate the robustness and generalizability of the proposed solution, the system was tested across three distinct piano models and camera configurations for each of which detected candidate and pressed keys are illustrated.